

COMP1531 • MAJOR PROJECT

# Unigotchi — Iteration 0

Requirements · Interface design · Stubs · Teamwork · Demo

# design before you build

# What Iteration 0 is (and isn't)

*A stub is a promise about the interface — **don't implement the mechanics yet.***

## THIS ITERATION

- Understand the requirements
- Design the function interfaces
- Set up how the team works together
- Write stub functions only

## DELIVERABLES

`contract.md`

meeting minutes

issue board + merge requests

`features.md`

`data.md`

**stub functions only**

# Starter code structure

```
project-backend/  
  contract.md  
  minutes-template.md  
  features.md  
  data.md  
  src/  
    auth.js  course.js  
    dataStore.js  exam.js  
    other.js  simulation.js  
  universes/  
    universe1..4.json
```

**contract.md** — team agreement: meetings, comms, conflict

**features.md** — main design doc: feature rows + interface table

**data.md** — proposed data structure (1 user + 1 sim)

**src/\*.js** — home for prescribed + designed stubs

**universes/\*.json** — example data to inspect while designing

**The spec is the source of truth — not the examples.**

# Reading a user story + acceptance criteria

**Story 1** *As a student, I want to see all of my Unigotchi simulation instances, so that I can choose which simulation to inspect or continue.*

## ANATOMY

**“As a student”** who needs this?

**“I want to see all...”** what capability?

**“so that I can choose...”** why it matters

**Acceptance criteria** how we know the design satisfies it

## WHICH PRESCRIBED FUNCTIONS HELP?

```
simulationCreate(studentId, universeData)
simulationStatus(studentId, simulationId)
```

Neither returns ALL simulations for a student.

**Prescribed interfaces must not change.**

**Need new behaviour? Design a NEW function.**

# From story to design • feature + interface

## 1 — FEATURE DESIGN ROW (features.md)

List simulation instances | Story 1 | Retrieve all simulations belonging to a student so they can choose which to inspect | Student records, simulation IDs, status, elapsed time

## 2 — FUNCTION INTERFACE

### **simulationList(studentId)**

```
{
  simulations: [
    {
      simulationId: 1,
      simulationStatus: 'ACTIVE',
      timeElapsed: 0,
    }
  ]
}
```

### **DESIGN TRADE-OFFS**

- simulationId, required by the AC
- status + timeElapsed - useful summary
- NOT full course/exam data - this is a list/summary
- Want detail? call simulationStatus later

## Step 3 — the stub (src/simulation.js)

```
// Return a summary list of all simulation
// instances belonging to a student.
function simulationList(studentId) {
  return {
    simulations: [
      {
        simulationId: 1,
        simulationStatus: 'ACTIVE',
        timeElapsed: 0,
      }
    ],
  };
}
```

**A stub proves the SHAPE of a function, not the logic.**

- No real behaviour
- Just a sample object matching the interface

# GitLab issues & merge requests

## ISSUE

[Iteration 0] Design and stub simulationList

## BRANCH

iter0-feat-simulation-list-stub

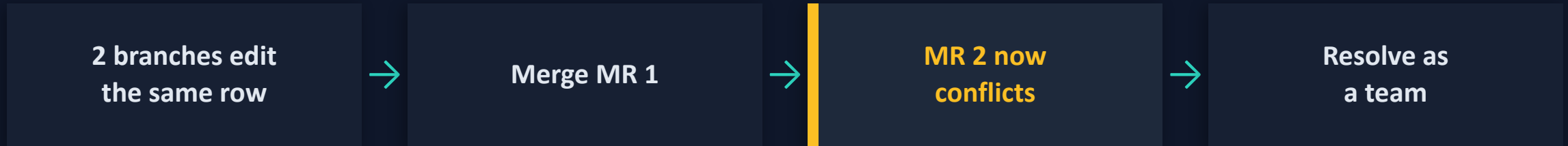
## MR TITLE

Add Story 1 simulation list design and stub  
MR BODY → Closes #<issue>

## GOOD PRACTICE

- One issue per small task
- One branch per issue / task
- MR links to its issue
- Someone ELSE reviews it
- Creator merges after approval
- No giant all-in-one MR

# Merge conflict demo



```
<<<<<< HEAD          your branch
| Simulation summaries | Story 1 | ...
=====
| List simulation instances | Story 1 | ...
>>>>>> origin/master    already merged
```

**HEAD** = your branch · **origin/master** = what's merged · **you decide the final content — never leave the markers in.**



# Teamwork advice & wrap-up

## FIVE HABITS

- Work small - one task per issue & MR
- Review meaningfully (not “looks good”)
- Never modify prescribed interfaces
- Keep master clean - it's what you submit
- Resolve conflicts as a team

## A USEFUL REVIEW COMMENT

✗ *“looks good”*

✓ “The interface row says simulationList returns simulationStatus, but the stub returns status. Can we make the names match?”

## THE WHOLE PATH

# Requirement → design → stub → MR → review → merge

1

Story: student wants to see all their simulations

2

AC: each needs a simulationId + useful summary

3

simulationStatus inspects one - can't list all

4

Prescribed functions must not be modified

5

So design simulationList(studentId)

6

Add feature + interface rows to features.md, make issue

7

Add the stub in src/simulation.js

8

Open a a merge request

9

Someone else reviews it

10

Merge into master

# Demo

1

INDIVIDUAL

## Your contribution

- Stub
- Merge request
- Issue board

2

GROUP

## Function design

- AC -> Function design
- Interface feedback and review
- How does it use the datastore?
- Stub review